

Helix Thready — Frequently Asked Questions

Table of contents

- 1. General
- 2. Install & configuration
- 3. Messengers & channels
- 4. Processing & categories
- 5. Search
- 6. Assets & downloads
- 7. Accounts, roles & security
- 8. Clients & SDKs
- 9. Data, backup & compliance

Rev	Date	Author	Change
1	2026-07-21	swarm (user-guides)	Initial FAQ
2	2026-07-22	swarm (user-guides, Pass 3)	Depth pass: corrected the headless sign-in var to the VERIFIED <code>HERALD_MTPROTO_*</code> names; added quickstart, env-var, and DR FAQ entries

Short answers with links to the authoritative guide. Answers are grounded in the technology decision matrix and the [gap register](#); where a feature depends on a scaffold/ `[BUILD-NEW]` module, the answer says so.

Table of contents

- 1. General
- 2. Install & configuration
- 3. Messengers & channels
- 4. Processing & categories
- 5. Search
- 6. Assets & downloads
- 7. Accounts, roles & security
- 8. Clients & SDKs
- 9. Data, backup & compliance

1. General

Q: What is Helix Thready? It connects to messaging platforms (Telegram, then Max, then more) with your private accounts, reads whole threads, processes each post through AI recipes (Skills) selected by hashtags/content type, generates research and downloadable assets, and makes everything searchable by meaning. See [index.md](#) and [end-user-manual.md](#).

Q: Is this the shipped product? No — this is the **zero version** documentation, written before implementation. It documents the system as decided in the decision matrix. Where a subsystem is a scaffold/stub, the guides say so.

Q: Who makes it? Helix Development. The footer slogan “*Made with love ♥ by Helix Development*” appears on every surface and generated document, and persists even under per-account white-labeling.

Q: I just want to try it — what’s the fastest path? Read [quickstart.md](#): a ~5-minute per-surface on-ramp (Web, CLI, TUI, SDK, Mobile) plus the one-time Admin bootstrap, each linking to its deep guide.

Q: Where is the full list of environment variables? [configuration.md](#), whose **Appendix A** is a single one-row-per-variable index (name · purpose · default · scope · example) and **Appendix B** has worked `.env` files for dev/staging/production.

2. Install & configuration

Q: What do I need to run it? Rootless Podman, Go 1.26.x (to build), PostgreSQL 16+ with pgvector, and a llama.cpp/HelixLLM endpoint. See [installation.md](#) §1.

Q: Where do settings live? In a gitignored `.env`, with values also readable from `~/.bashrc` / `~/.zshrc` and `$HOME/api_keys.sh`; missing sources are skipped silently, never logged. The full list is [configuration.md](#).

Q: I set a var in `.env` but nothing changed. A shell `export` overrides `.env` (process-env wins). Also, **secrets are not hot-reloaded** — restart after rotating them. [troubleshooting.md](#) §9.

Q: Docker? No — **rootless Podman only** `[CONSTITUTION §11.4.76]`.

3. Messengers & channels

Q: Which messengers work today? **Telegram** (via the `gotd/td` MTPROTO user client, being promoted from Herald’s `qaherald` harness). **Max is not available yet** — the adapter is `[BUILD-NEW]` `[GAP: 3]`. Plan around Telegram for the zero version.

Q: Interactive or headless sign-in? Both. Interactive prompts for the login code/2FA; headless (`THREADY_MESSENGER_SIGNIN_MODE=noninteractive`) reads the VERIFIED `HERALD_MTPROTO_*` variables (`APP_ID` / `APP_HASH` / `PHONE` / `PASSWORD` / `SESSION_FILE`) from the environment — the user client that reads history. The Bot-API path (`HERALD_TGRAM_BOT_TOKEN`) only posts replies. [installation.md](#) §7, [configuration.md](#) §9.

Q: What are the real messenger env-var names? For Telegram reading: `HERALD_MTPROTO_APP_ID`, `HERALD_MTPROTO_APP_HASH`, `HERALD_MTPROTO_PHONE`, `HERALD_MTPROTO_PASSWORD` (2FA), `HERALD_MTPROTO_SESSION_FILE` — VERIFIED from `vasic-digital/herald/.env.example`. Early drafts used `HERALD_TELEGRAM_*`; those were wrong and have been corrected.

Q: Does it read replies too? Yes — Thready assembles the **complete post** = root + the full chain of organic human replies (excluding the system's own replies). Hashtags added in a reply still count. [end-user-manual.md §3](#).

Q: Can I just add a channel and let it figure out the content? Yes — auto-recognition detects the thread's content type and how to process it; you can override per channel.

4. Processing & categories

Q: How does it decide what to do with a post? By its hashtags + content type, mapped to Skills. Missing/weak tags trigger indirect determination (GitHub→research, torrent→download, ...); still unclassified → generic ingest + manual-review queue, never dropped. [end-user-manual.md §5](#).

Q: A post has several hashtags — which wins? None — they're **additive**. All matching recipes run, ordered `download > convert > analyze > research > reply`. A `#Research #Video` post both downloads the video and does deep research.

Q: Will the same post be processed twice under an event storm? No. Each post is claimed exactly once via a Postgres lock (single-claim idempotency) [\[CONSTITUTION §11.4.176\]](#).

Q: Can I re-process a post? Yes — `thready post reprocess <id>` (full refresh) or `thready post retry <id> --step <step>` (one step). Client → REST → System.

Q: What do the status replies contain? Success/failure, metrics, and asset references, posted from the Robot or User account (`THREADY_REPLY_ACCOUNT`).

5. Search

Q: How does search work? Semantic (by meaning) over both original posts and generated materials, via pgvector + a llama.cpp embedding model — the in-house “Lumen-style” capability. Target < 500 ms. [end-user-manual.md §6](#).

Q: My search results are irrelevant. Almost always the HelixLLM **default hash embedder** [\[GAP: 1\]](#). Set `HELIX_EMBEDDING_PROVIDER=llama`, ensure `THREADY_EMBEDDING_DIM` matches the model, reindex, and restart. [troubleshooting.md §5](#).

Q: Which vector DB? pgvector (cosine), co-located in Postgres. Qdrant/Pinecone/Milvus sit behind the same interface but are **unverified** [\[GAP: 8\]](#) — stay on pgvector.

6. Assets & downloads

Q: Are download links direct file paths? No — every asset link resolves through the **Asset Service** (built on Catalogizer) with auth/RBAC. Raw original + a web-optimized `-web` rendition are kept. [end-user-manual.md §7](#).

Q: A download never finishes. Depends on the path: MeTube is **poll-only** `[GAP: 5]` (finishes but notifies slowly), the generic Download Manager for direct URLs **doesn't exist yet** `[GAP: 4]`, and Boba callbacks are being standardized. [troubleshooting.md §6](#).

Q: Comic/screenshot text isn't extracted. VisionEngine has **no OCR engine** `[GAP: 2]`; it falls back to LLM-vision until the Tesseract/PaddleOCR adapter `[BUILD-NEW]` lands.

Q: A physical file link broke. `thready asset reheal <id>` re-downloads it.

7. Accounts, roles & security

Q: What are the roles? Root Admin (owns everything, exactly one), Account Admin (owns one account), Standard User (consumer). [root-admin-guide.md](#), [account-admin-guide.md](#).

Q: Can one person be in multiple accounts? Yes — with different roles per account; a user can create their own account and become its admin.

Q: Is MFA required? TOTP is **mandatory for Root and Account Admins**, optional for Standard Users (`THREADY_MFA_REQUIRED_TIERS`).

Q: How are passwords/tokens protected? Argon2id password hashing (≥ 12 , breach-checked); AES-256-GCM at rest; TLS 1.3 in transit. JWT is HS256 today, moving to RS256/EdDSA for multi-service `[GAP: 10]`.

Q: Can mobile clients be used with real accounts now? No — `Security-KMP` mobile secure storage is an **in-memory stub** `[GAP: 7]`; mobile is evaluation-only until Android Keystore / iOS Keychain are implemented. [mobile-guide.md §3](#).

8. Clients & SDKs

Q: Which clients exist? Web (primary), CLI, TUI, Desktop (Tauri), Mobile (native + KMP). Web + CLI are first priority. All use the same REST `/v1` + event bus.

Q: Is the CLI as capable as the Web? Yes — “everything possible from the Web works from the CLI/TUI” (headless, pipeline-friendly). [cli-reference.md](#).

Q: Which SDK language? Go is **primary**; TS/Python/Java-Kotlin are High priority; others follow. Generated from OpenAPI 3.1 + Protobuf. [sdk-quickstart.md](#).

Q: Which languages is the UI available in? The UI is fully localized in **English** (`en`), **Russian** (`ru`), and **Serbian Cyrillic** (`sr-Cyrl`) via HelixTranslate + `digital.vasic.i18n` (Q35). Set the default with `THREADY_DEFAULT_LOCALE`; users can switch in-app, with an explicit choice plus system default. **Post**

content is stored in its **original** language (not auto-translated) with on-demand translation available; generated research is English-primary with on-demand translation. [configuration.md §15](#).

9. Data, backup & compliance

Q: How long is data kept? Indefinitely by default, with per-account overrides (accounts may shorten). [root-admin-guide.md §6](#).

Q: What's the backup policy? Daily full + hourly DB incrementals; assets daily snapshot; RPO $\approx$ 1 h, RTO $\approx$ 4 h with a documented restore runbook. [root-admin-guide.md §10](#).

Q: Is it GDPR-certified? No — compliance is **minimal/internal** for the MVP, but the design is GDPR-aware (erasure/export hooks exist). Certification is deferred [\[OPEN\]](#).

Q: Where are the logs / how do I monitor it? logrus + ClickHouse (logs), OpenTelemetry (traces), Prometheus + Grafana (metrics) — the in-house stack, **not** ELK/Loki/Datadog. [troubleshooting.md §10](#).

Made with love ♥ by Helix Development.